Optimisation des fonctions d'activation pour les réseaux de neurones
Étude appliquée au *Full Adder* et implémentation CORDIC sur FPGA

Fatima-Ezzahra Ahchouch Lina Benlahsar

Mars 2026

Sommaire

1. Le Full Adder comme base de modélisation
2. Neurone formel et apprentissage
3. Fonctions d'activation et cas logiques
4. Études préalables : ET et XOR
5. Réseau de neurones pour le Full Adder
6. Implémentation matérielle des activations
7. Méthodes classiques d'approximation
8. L'algorithme CORDIC
9. Conclusion générale

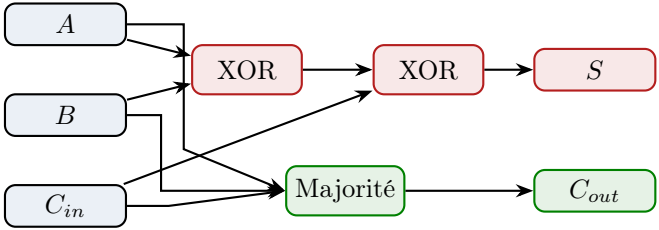
Le Full Adder : table, équations et décomposition logique

Table de vérité

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Équations logiques

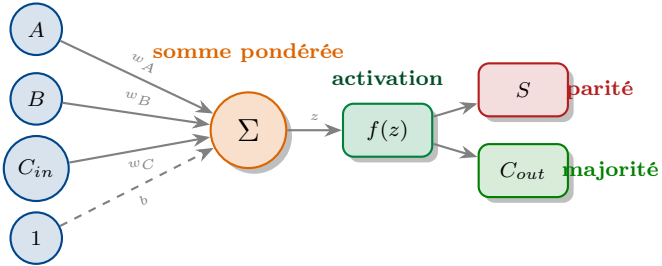
$$S = A \oplus B \oplus C_{in} \qquad C_{out} = AB + AC_{in} + BC_{in}$$



Lecture mathématique

La sortie S contient la difficulté du XOR non séparable, tandis que C_{out} est une fonction seuil de majorité : les deux sorties exigent donc une architecture capable de combiner décision binaire et non-linéarité.

Structure d'un neurone artificiel appliquée au Full Adder



Modèle mathématique

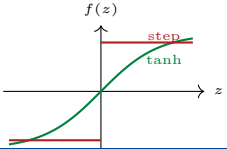
$$z = w_A A + w_B B + w_C C_{in} + b, \qquad y = f(z)$$

Commentaire

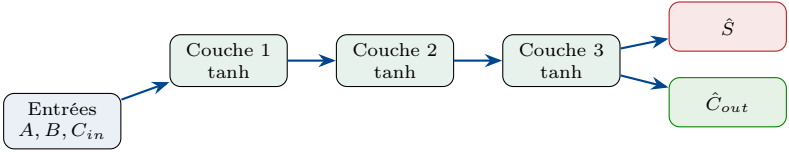
Un neurone isolé réalise une séparation par seuil. Il modélise naturellement une décision simple ou une majorité partielle, mais pas la parité du bit somme.

Conséquence

La sortie $S = A \oplus B \oplus C_{in}$ nécessite des couches cachées et une activation non linéaire ; C_{out} est plus proche d'une fonction de seuil.



Propagation avant : calcul de $(\hat{S}, \hat{C}_{out})$



Accumulation pondérée : $z = Wx + b$

Calcul direct

La propagation avant applique successivement les poids, les biais et les activations afin d'estimer les deux sorties :

$$(A, B, C_{in}) \longrightarrow h^{(1)} \longrightarrow h^{(2)} \longrightarrow (\hat{S}, \hat{C}_{out}).$$

Objectif de la propagation

Chaque ligne de la table du Full Adder doit être projetée vers les deux bits logiques attendus.

Rétropropagation : correction des poids

Erreur globale

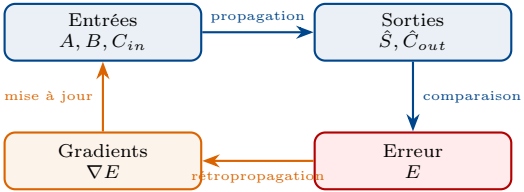
$$E = \frac{1}{2}\left[(S - \hat{S})^2 + (C_{out} - \hat{C}_{out})^2\right]$$

Mise à jour

$$w \leftarrow w - \eta \frac{\partial E}{\partial w}, \qquad b \leftarrow b - \eta \frac{\partial E}{\partial b}$$

Condition essentielle

La rétropropagation nécessite une activation différentiable dans les couches cachées : tanh ou sigmoïde. La fonction step est utilisée en sortie ou pour une construction logique exacte.

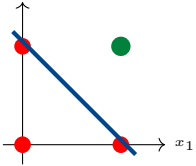


Lecture Full Adder

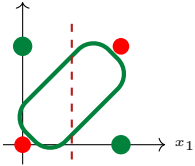
Les couches cachées ajustent leurs frontières internes afin de reproduire simultanément la parité S et la majorité C_{out}.

Pourquoi une fonction d'activation ?

AND : séparable linéairement



XOR : non séparable



Sans non-linéarité

Des couches linéaires empilées restent équivalentes à une seule transformation linéaire ; elles ne résolvent pas XOR.

Avec activation

Une activation comme tanh courbe l'espace de représentation et permet des frontières de décision composées.

Cas Full Adder

C_{out} relève d'une majorité, mais $S = A \oplus B \oplus C_{in}$ impose non-linéarité et couches cachées.

Les fonctions d'activation cibles : step, sigmoïde et tanh

Step

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

- Décision logique nette.
- Non différentiable.
- Adaptée au seuillage final.

Sigmoïde

$$\sigma(x) = \frac{1}{1 + e^{-x}},$$

- Sortie dans $(0, 1)$.
- Gradient maximum 0.25.
- Coûteuse en exponentielle.

Tangente hyperbolique

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}},$$

- Sortie dans $(-1, 1)$.
- Centrée en zéro.
- Gradient maximum 1.

Relation fondamentale

$$\tanh(x) = 2\sigma(2x) - 1 \qquad \Longleftrightarrow \qquad \sigma(x) = \frac{1 + \tanh(x/2)}{2}$$

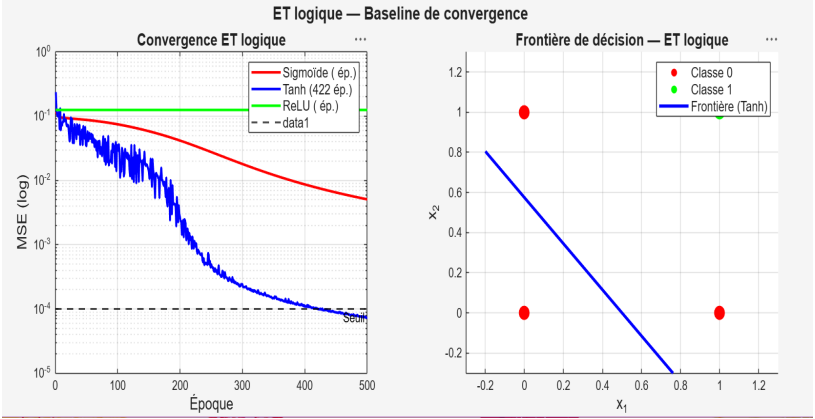
Étude préalable : ET logique comme baseline de convergence

Fonction linéairement séparable

- Un neurone à seuil peut réaliser la fonction ET.
- Les frontières de décision sont simples.
- La convergence sert à valider la chaîne d'apprentissage.

Lien avec C_{out}

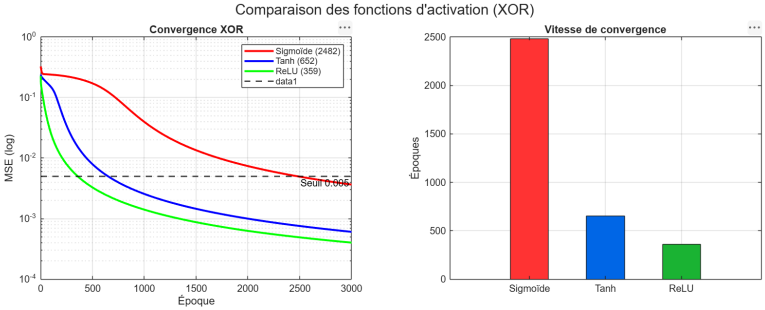
La retenue du Full Adder repose sur des produits logiques AB, AC_{in}, BC_{in} puis une majorité.



Étude de cas : XOR, cœur non linéaire de la somme

Non-séparabilité

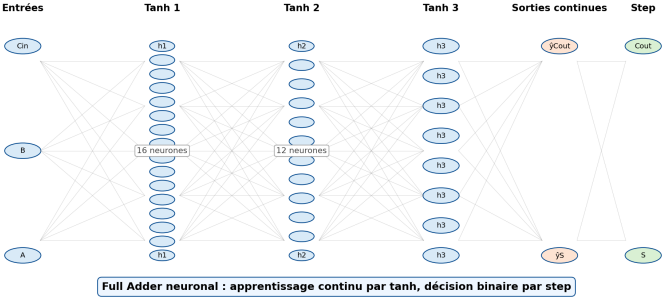
- XOR ne peut pas être séparé par une seule droite.
- Une couche cachée crée des frontières intermédiaires.
- Le bit S du Full Adder est une parité : $A \oplus B \oplus C_{in}$.



Conclusion expérimentale

Tanh converge plus rapidement que sigmoïde grâce à son gradient plus grand et à sa sortie centrée en zéro.

Architecture Full Adder : combinaison tanh + step

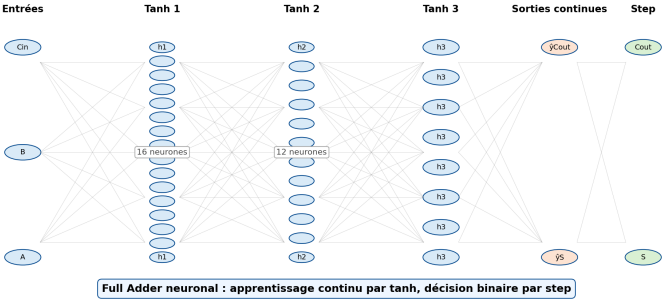


Architecture profonde proposée

$$\boxed{3 \to 16 \to 12 \to 8 \to 2 \to \text{step}}$$

entrées A, B, C_{in} $\to$ couches cachées tanh $\to$ sorties continues $\to$ décision binaire

Architecture Full Adder : combinaison tanh + step



Pourquoi plusieurs couches cachées ?

La parité à trois entrées est plus complexe qu'un XOR simple. La profondeur factorise les combinaisons élémentaires, les motifs XOR puis la fusion vers S et C_{out}.

Rôle des activations

Tanh assure l'apprentissage différentiable dans les couches cachées. **Step** transforme les deux sorties continues en bits logiques 0/1.

Full Adder : réseau tanh profond puis seuillage step

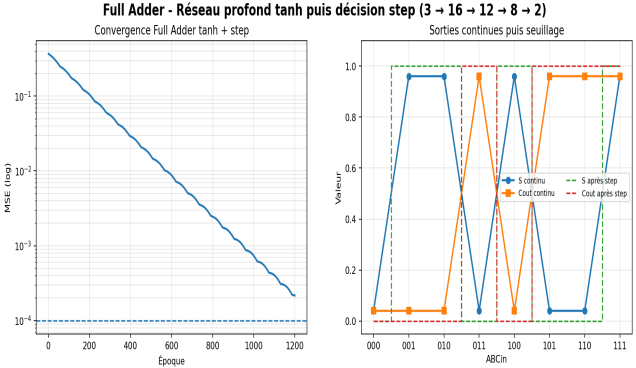
Architecture entraînable

$$\boxed{3 \to 16 \to 12 \to 8 \to 2}$$

- Tanh dans toutes les couches cachées.
- Sorties continues converties par step : $y \geq 0.5 \Rightarrow 1$, sinon 0.
- Optimisation par rétropropagation sur les 8 lignes du Full Adder.

Lien vers le FPGA

La présence de tanh rend nécessaire une implémentation matérielle efficace de cette activation.



Problématique matérielle : pourquoi optimiser tanh et σ ?

Calcul direct coûteux sur FPGA

- $\sigma(x) = \dfrac{1}{1+e^{-x}}$ nécessite exponentielle et division.
- $\tanh(x) = \dfrac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$ nécessite aussi des opérations coûteuses.
- Les blocs DSP et les ressources logiques deviennent rapidement limitants.

Trois approches étudiées

1. **LUT** : table de correspondance.
2. **Taylor** : approximation polynomiale.
3. **CORDIC** : additions et décalages binaires.

Critères

Précision, stabilité sur l'intervalle utile, faible mémoire, absence de multiplieurs DSP.

Objectif : remplacer les exponentielles par une architecture shifts + additions

Méthode LUT : simple, mais pas forcément optimale

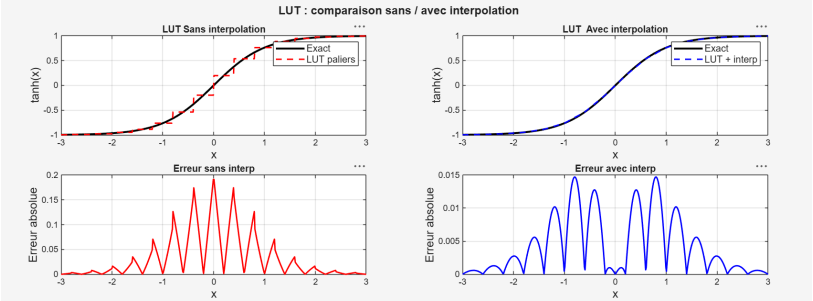
Principe

La fonction est échantillonnée puis stockée en ROM :

$$x_i = x_{min} + i\Delta x,$$

f(x)≈ LUT[i].

- Sans interpolation : erreur en dents de scie.
- Avec interpolation : précision améliorée.
- Interpolation : coût d'un multiplieur.



Développement de Taylor : précis localement, fragile globalement

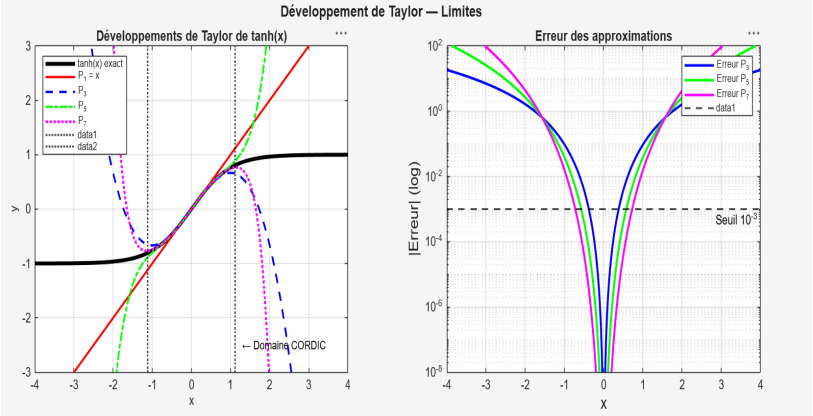
Approximation polynomiale

$$P_3(x) = x - \frac{x^3}{3}$$

$$P_5(x) = x - \frac{x^3}{3} + \frac{2x^5}{15}$$

Limites

- Précision locale autour de zéro.
- Divergence pour les grands $|x|$.
- Multiplications coûteuses en FPGA.



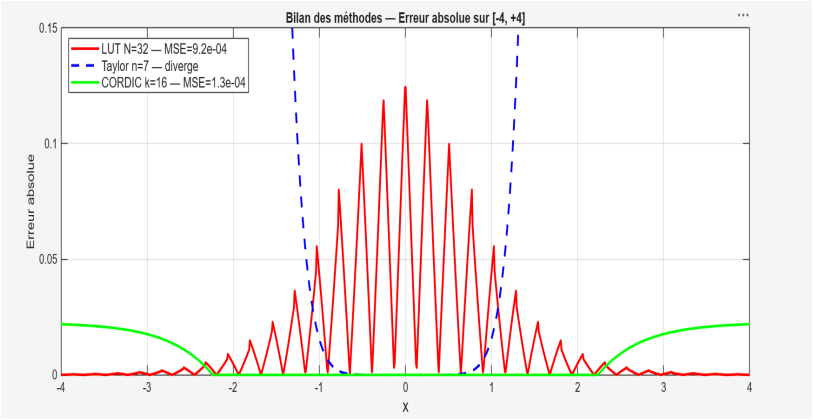
Comparaison des méthodes d'approximation sur tanh

Lecture des erreurs

- LUT : stable mais quantifiée.
- Taylor : bonne près de 0, instable hors zone locale.
- CORDIC : erreur très faible et plus uniforme.

Conclusion

Une solution matérielle robuste doit éviter à la fois la mémoire excessive, les multiplications et la divergence hors domaine.



Bilan comparatif : choix de la méthode CORDIC

Méthode	**MSE**	**Stabilité**	**DSP ?**	**Analyse**
LUT $N = 32$ sans interp.	$\approx 10^{-3}$	Stable	Non	Simple, mais effet escalier
LUT $N = 16$ avec interp.	$\approx 2 \times 10^{-5}$	Stable	Oui	Précise, mais nécessite un multiplieur
Taylor $n = 7$	$\approx 10^{-4}$	Diverge	Oui	Peu robuste hors voisinage de zéro
CORDIC $k = 16$	$\approx \mathbf{10^{-8}}$	**Uniforme**	**Non**	**Solution retenue pour** tanh **et** σ

Solution retenue : CORDIC hyperbolique

CORDIC : définition et principe général

Définition

CORDIC (*Coordinate Rotation Digital Computer*) calcule des fonctions trigonométriques et hyperboliques par une succession d'additions, de soustractions et de décalages binaires.

Astuce matérielle

$$a \times 2^{-k} \equiv a \gg k$$

La multiplication par une puissance de deux devient un simple décalage à droite.

Deux modes

- **Circulaire** : sin, cos.
- **Hyperbolique** : sinh, cosh, tanh.

Relation utilisée

$$\sigma(x) = \frac{1 + \tanh(x/2)}{2}$$

Un seul bloc hyperbolique permet donc de produire tanh et σ.

CORDIC circulaire ($m = +1$) : sin et cos

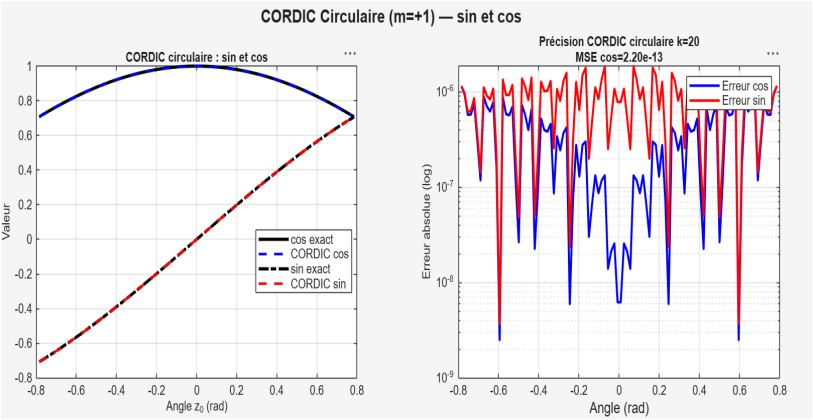
Équations itératives

$$\begin{cases} x_{k+1} = x_k - d_k 2^{-k} y_k \\ y_{k+1} = y_k + d_k 2^{-k} x_k \\ z_{k+1} = z_k - d_k \arctan(2^{-k}) \end{cases}$$

$$d_k = \text{sign}(z_k)$$

Résultat

Après n itérations :
$x_n \approx \cos(z_0)$, $y_n \approx \sin(z_0)$.



CORDIC hyperbolique ($m = -1$) : tanh et σ

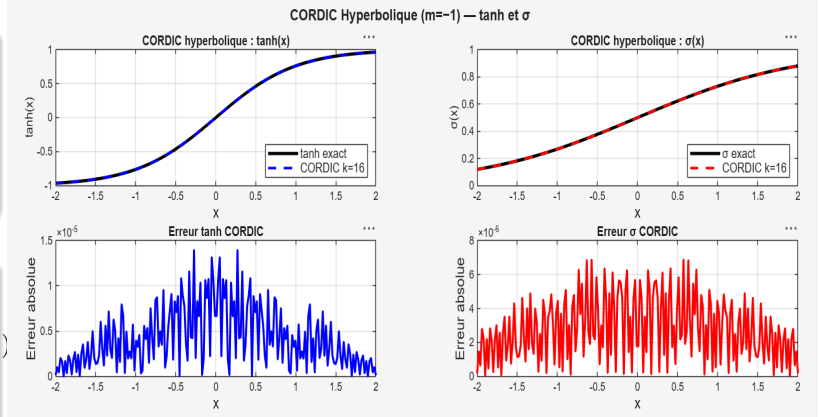
Équations itératives

$$\begin{cases} x_{k+1} = x_k + d_k 2^{-k} y_k \\ y_{k+1} = y_k + d_k 2^{-k} x_k \\ z_{k+1} = z_k - d_k \operatorname{atanh}(2^{-k}) \end{cases}$$

Sortie

$$x_n \approx \cosh(z_0), \qquad y_n \approx \sinh(z_0)$$

$$\boxed{\tanh(z_0) = \frac{y_n}{x_n}}$$



Pourquoi répéter certaines itérations en mode hyperbolique ?

Problème

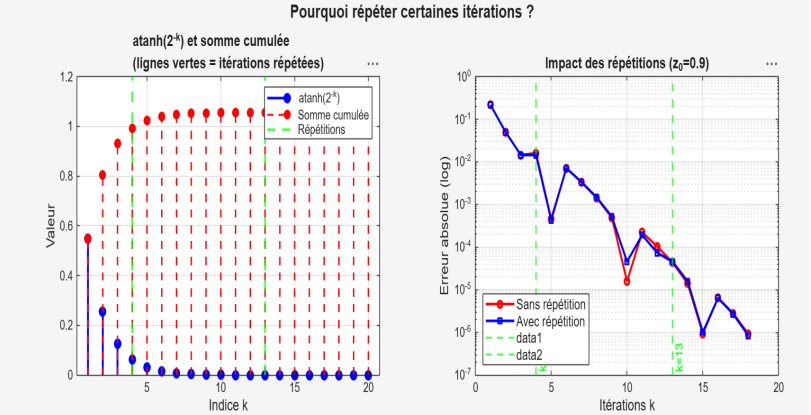
En mode hyperbolique, certaines corrections élémentaires ne suffisent pas à garantir la convergence sur toute la plage utile.

Solution

Les itérations critiques sont répétées :

$$k = 4,\ 13,\ 40,\ 121, \ldots$$

avec $k_{i+1} = 3k_i + 1$.



Conséquence

Pour $k = 16$, les répétitions $k = 4$ et $k = 13$ conduisent à 18 passes.

Choix du domaine de travail : influence sur le gradient de tanh

Contrainte

Le CORDIC hyperbolique converge naturellement sur un domaine limité :

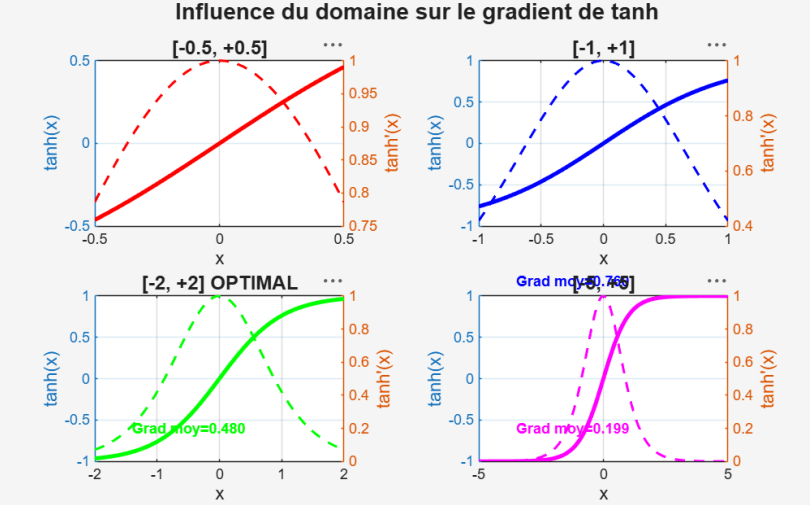
$$|z_0| < \sum_{k=1}^{n} \operatorname{atanh}(2^{-k}) \approx 1.118.$$

Choix pratique

Pour traiter $[-2, +2]$, on normalise l'entrée :

$$z' = z/2 \in [-1, +1].$$

Ce choix conserve une courbure utile sans saturation excessive.



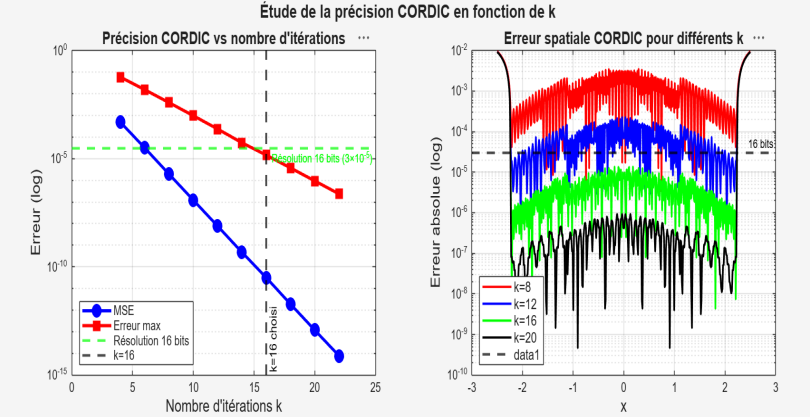
Précision CORDIC en fonction du nombre d'itérations

Règle pratique

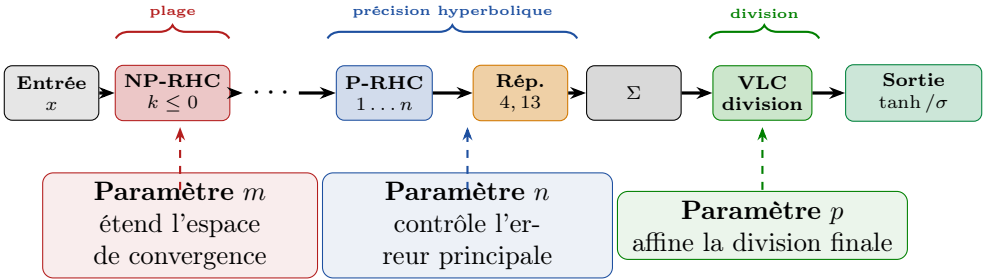
Une itération CORDIC apporte approximativement un bit de précision supplémentaire.

Choix de $k = 16$

- Cohérent avec une représentation fixe 16 bits.
- Erreur du même ordre que la résolution 2^{-15}.
- Compromis précision / latence / ressources.



Architecture CORDIC paramétrée : étages et flux de données



Conclusion

Synthèse finale

- Le Full Adder met en évidence la nécessité de la non-linéarité pour modéliser des fonctions complexes comme le XOR.
- L'utilisation de tanh permet l'apprentissage via un réseau multicouche, tandis que le seuillage final restitue la logique binaire.
- Enfin, l'algorithme CORDIC offre une implémentation matérielle efficace de cette activation sur FPGA.

Merci pour votre attention

Questions ?